N.Moksha Sai Assignment 2

Day 2 – Problem Solving Techniques and Data Structures – II

Set 1 – E-Commerce Catalogue and Traffic Analysis

Task 1: Binary Search Tree of 15 product categories with traversals

Code:

```
class Node:
    def __init__(self, name):
        self.name = name
        self.left = None
        self.right = None

def insert(root, name):
    if root is None:
        return Node(name)
    if name < root.name:
        root.left = insert(root.left, name)
    elif name > root.name:
        root.right = insert(root.right, name)
    return root

def search(root, name):
    if root is None:
        return False
    if name == root.name:
        return True
    if name < root.name:
        return search(root.left, name)
    return search(root.right, name)

def inorder(root, out):
    if root:
        inorder(root.left, out)
        out.append(root.name)
        inorder(root.right, out)

def preorder(root, out):
    if root:
        out.append(root.name)
        preorder(root.left, out)
        preorder(root.right, out)

def postorder(root, out):
    if root:
        postorder(root.left, out)
        postorder(root.right, out)
        out.append(root.name)

categories = ["Electronics", "Books", "Clothing", "Toys", "Sports",
              "Grocery", "Furniture", "Beauty", "Garden", "Music",
              "Pets", "Shoes", "Watches", "Automotive", "Jewelry"]

root = None
for c in categories:
    root = insert(root, c)

a = []
inorder(root, a)
print("Inorder  :", ", ".join(a))
b = []
preorder(root, b)
print("Preorder :", ", ".join(b))
```

```
c = []
postorder(root, c)
print("Postorder:", ", ".join(c))
print("Search Toys:", search(root, "Toys"))
print("Search Cars:", search(root, "Cars"))
```

Output:

```
Inorder  : Automotive, Beauty, Books, Clothing, Electronics, Furniture, Garden,
    Grocery, Jewelry, Music, Pets, Shoes, Sports, Toys, Watches
Preorder : Electronics, Books, Beauty, Automotive, Clothing, Toys, Sports,
    Grocery, Furniture, Garden, Music, Jewelry, Pets, Shoes, Watches
Postorder: Automotive, Beauty, Clothing, Books, Garden, Furniture, Jewelry,
    Shoes, Pets, Music, Grocery, Sports, Watches, Toys, Electronics
Search Toys: True
Search Cars: False
```

Explanation:

I used a BST because the categories stay in sorted order and searching is quick. Inorder (left, root, right) gives the categories in alphabetical order. Preorder visits the root first and postorder visits the root last. The names are compared as words, so smaller words go to the left.

Task 2: Hash table with chaining for SKU look-up, compared with the built-in hash map

Code:

```
import time

class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, sku):
        return sku % self.size

    def insert(self, sku, name):
        index = self.hash_function(sku)
        self.table[index].append((sku, name))

    def search(self, sku):
        index = self.hash_function(sku)
        for key, name in self.table[index]:
            if key == sku:
                return name
        return None

# small example to show a collision
small = HashTable(5)
small.insert(101, "Phone")
small.insert(106, "Laptop")
small.insert(203, "Mouse")
print("101 % 5 =", 101 % 5, " and 106 % 5 =", 106 % 5, "(collision)")
print("Bucket 1:", small.table[1])
print("Search 106:", small.search(106))
print("Search 999:", small.search(999))

# compare with built-in dictionary for 100,000 SKUs
skus = list(range(100000, 200000))

my_table = HashTable(10007)
for s in skus:
    my_table.insert(s, "item")

built_in = {}
for s in skus:
    built_in[s] = "item"
```

```
start = time.time()
for s in skus:
    my_table.search(s)
t1 = time.time() - start

start = time.time()
for s in skus:
    built_in.get(s)
t2 = time.time() - start

print("My hash table look-up time :", round(t1, 4), "seconds")
print("Built-in dict look-up time :", round(t2, 4), "seconds")
```

Output:

```
101 % 5 = 1  and 106 % 5 = 1 (collision)
Bucket 1: [(101, 'Phone'), (106, 'Laptop')]
Search 106: Laptop
Search 999: None
My hash table look-up time : 0.0297 seconds
Built-in dict look-up time : 0.0059 seconds
```

Explanation:

The hash function is SKU % size. SKUs 101 and 106 both give index 1 when the size is 5, so this is a collision. With chaining, both are kept in a list inside that bucket, and a search checks the items of that list one by one. For 100,000 SKUs the built-in dictionary was faster in my run, because it is written in C and is well optimised. The exact time can change a little every time the program runs.

Task 3: Microservice deployment order using a graph (DFS) and circular dependency check

Code:

```
# "A -> B" means A needs B, so B must be deployed first
graph = {
    "cart": ["pricing", "payment"],
    "pricing": ["inventory"],
    "payment": ["database"],
    "inventory": ["database"],
    "database": []
}

def has_cycle(graph):
    state = {}   # 1 = being visited, 2 = finished
    def dfs(node):
        state[node] = 1
        for nxt in graph[node]:
            if state.get(nxt) == 1:
                return True
            if nxt not in state and dfs(nxt):
                return True
        state[node] = 2
        return False
    for node in graph:
        if node not in state and dfs(node):
            return True
    return False

def deploy_order(graph):
    visited = set()
    order = []
    def dfs(node):
        visited.add(node)
        for nxt in graph[node]:
            if nxt not in visited:
```

```
                dfs(nxt)
            order.append(node)      # added after its dependencies
        for node in graph:
            if node not in visited:
                dfs(node)
        return order

if has_cycle(graph):
    print("Circular dependency found")
else:
    print("No circular dependency")
    print("Deployment order:", ", ".join(deploy_order(graph)))

# now add a circular dependency: database needs cart
graph["database"].append("cart")
if has_cycle(graph):
    print("After adding database -> cart: circular dependency found")
```

Output:

```
No circular dependency
Deployment order: database, inventory, pricing, payment, cart
After adding database -> cart: circular dependency found
```

Explanation:

Each service is a node and an arrow A -> B means A needs B. In DFS, a service is added to the list only after all the services it needs are added, so the list is a valid deployment order (database first, cart last). To find a circular dependency, DFS marks nodes that are currently being visited. If DFS reaches such a node again, there is a cycle, and then no valid order is possible.

Task 4: Busiest continuous 3-hour window using sliding window

Code:

```
# requests per minute for each hour of the day (hour 0 to hour 23)
rpm = [120, 90, 80, 70, 60, 65, 100, 180, 260, 340, 400, 420,
       450, 430, 380, 350, 370, 410, 460, 500, 430, 300, 200, 150]
k = 3

window_sum = sum(rpm[:k])
best_sum = window_sum
best_start = 0

for i in range(k, len(rpm)):
    window_sum = window_sum + rpm[i] - rpm[i - k]
    if window_sum > best_sum:
        best_sum = window_sum
        best_start = i - k + 1

print("Busiest 3-hour window:", str(best_start) + ":00 to",
      str(best_start + k) + ":00")
print("Total of the 3 hourly values:", best_sum)
```

Output:

```
Busiest 3-hour window: 18:00 to 21:00
Total of the 3 hourly values: 1390
```

Explanation:

The first window sum is calculated once. After that, I move the window by adding the new hour and removing the old hour, so I do not add 3 numbers again every time. The values are the requests per minute of each hour, and the window with the biggest total is the busiest one (18:00 to 21:00).

Task 5: Pairs that add up to the gift-card value using two pointers

Code:

```
orders = [100, 200, 250, 300, 400, 500, 600, 700, 800, 900]
target = 1000

left = 0
right = len(orders) - 1
pairs = []

while left < right:
    total = orders[left] + orders[right]
    if total == target:
        pairs.append((orders[left], orders[right]))
        left += 1
        right -= 1
    elif total < target:
        left += 1
    else:
        right -= 1

print("Gift card value:", target)
print("Pairs found:", pairs)
```

Output:

```
Gift card value: 1000
Pairs found: [(100, 900), (200, 800), (300, 700), (400, 600)]
```

Explanation:

The list is already sorted, so one pointer starts at the left and one at the right. If the sum is smaller than the target, the left pointer moves right. If it is bigger, the right pointer moves left. If it matches, the pair is saved and both pointers move. It stops when the pointers meet. I assumed the order values are all different.

Decision Table

Problem	Data structure / pattern used	Reason
Category browsing (Task 1)	Binary Search Tree	Keeps data sorted; inorder gives sorted list; search is O(log n) on average.
SKU look-up (Task 2)	Hash table / HashMap	Gives the item directly from the key; look-up is O(1) on average.
Deployment order (Task 3)	Graph with DFS	Services depend on each other; DFS gives the order and finds cycles.
Busiest 3-hour window (Task 4)	Sliding window	Reuses the previous sum, so one pass is enough, O(n).
Gift-card pairs (Task 5)	Two pointers	List is sorted, so pairs are found in one pass, O(n).

Set 2 – Insurance Claims Processing

Task 1: BST of claim IDs with insert, search and the three traversals

Code:

```
class Node:
    def __init__(self, value):
        self.value = value
```

```
        self.left = None
        self.right = None

def insert(root, value):
    if root is None:
        return Node(value)
    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)
    return root

def search(root, value):
    if root is None:
        return False
    if value == root.value:
        return True
    if value < root.value:
        return search(root.left, value)
    return search(root.right, value)

def inorder(root, out):
    if root:
        inorder(root.left, out)
        out.append(root.value)
        inorder(root.right, out)

def preorder(root, out):
    if root:
        out.append(root.value)
        preorder(root.left, out)
        preorder(root.right, out)

def postorder(root, out):
    if root:
        postorder(root.left, out)
        postorder(root.right, out)
        out.append(root.value)

claim_ids = [450, 300, 600, 250, 350, 500, 700, 300]   # 300 is repeated

root = None
for cid in claim_ids:
    root = insert(root, cid)

a = []
inorder(root, a)
print("Inorder  :", a)
b = []
preorder(root, b)
print("Preorder :", b)
c = []
postorder(root, c)
print("Postorder:", c)
print("Search 350:", search(root, 350))
print("Search 999:", search(root, 999))

empty = []
inorder(None, empty)
print("Empty tree inorder:", empty)
```

Output:

```
Inorder  : [250, 300, 350, 450, 500, 600, 700]
Preorder : [450, 300, 250, 350, 600, 500, 700]
Postorder: [250, 350, 300, 500, 700, 600, 450]
Search 350: True
Search 999: False
```

```
Empty tree inorder: []
```

Explanation:

Inorder traversal gives the claims in sorted order. This is because in a BST everything on the left is smaller than the node and everything on the right is bigger, and inorder visits left, then the node, then right. A repeated ID (300) is ignored, so it appears only once. An empty tree gives an empty list.

Task 2: Detect duplicate or near-duplicate claims using hashing

Code:

```
def make_key(policy, date):
    # remove all spaces and make lower case
    p = "".join(policy.split()).lower()
    d = "".join(date.split()).lower()
    return p + "|" + d

def find_duplicates(claims):
    seen = set()
    duplicates = []
    for policy, date in claims:
        key = make_key(policy, date)
        if key in seen:
            duplicates.append((policy, date))
        else:
            seen.add(key)
    return duplicates

claims = [
    ("POL123", "2025-03-10"),
    ("pol123", "2025-03-10"),
    ("POL 123", "2025-03-10"),
    ("POL456", "2025-03-11"),
    ("POL456", "2025-03-12"),
    ("pol456 ", "2025-03-11"),
]

print("Duplicates:", find_duplicates(claims))
print("Empty input:", find_duplicates([]))
print("Same claim twice:", find_duplicates([("A1", "2025-01-01"),
                                            ("A1", "2025-01-01")]))
```

Output:

```
Duplicates: [('pol123', '2025-03-10'), ('POL 123', '2025-03-10'), ('pol456 ',
    '2025-03-11')]
Empty input: []
Same claim twice: [('A1', '2025-01-01')]
```

Explanation:

For every claim I make a key from the policy number and the date after removing spaces and changing to lower case. The keys are stored in a set (a hash table). If the key is already in the set, the claim is a duplicate. A collision happens when two different keys land in the same place in the table. Python's set handles this by itself: it looks for another free place and it also compares the real keys, so two different claims are never treated as the same.

Task 3: Claim approval stages as a directed graph: shortest path (BFS) and loop detection (DFS)

Code:

```
from collections import deque
```

```
stages = {
    "Registration": ["Assessment"],
    "Assessment": ["Surveyor", "Settlement"],
    "Surveyor": ["Settlement"],
    "Settlement": []
}

def shortest_path(graph, start, end):
    parent = {start: None}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node == end:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for nxt in graph[node]:
            if nxt not in parent:
                parent[nxt] = node
                queue.append(nxt)
    return None

def has_loop(graph):
    state = {}   # 1 = being visited, 2 = finished
    def dfs(node):
        state[node] = 1
        for nxt in graph[node]:
            if state.get(nxt) == 1:
                return True
            if nxt not in state and dfs(nxt):
                return True
        state[node] = 2
        return False
    for node in graph:
        if node not in state and dfs(node):
            return True
    return False

print("Registration to Settlement:",
      " -> ".join(shortest_path(stages, "Registration", "Settlement")))
print("Settlement to Registration:",
      shortest_path(stages, "Settlement", "Registration"))
print("Loop in stages?", has_loop(stages))

# send the claim back from Surveyor to Assessment (creates a loop)
stages["Surveyor"].append("Assessment")
print("Loop after adding Surveyor -> Assessment?", has_loop(stages))
```

Output:

```
Registration to Settlement: Registration -> Assessment -> Settlement
Settlement to Registration: None
Loop in stages? False
Loop after adding Surveyor -> Assessment? True
```

Explanation:

The stages are nodes and the arrows show which stage can come next. BFS visits the stages level by level, so the first time it reaches the end stage, the path found has the fewest steps. DFS finds a loop when it reaches a stage that is still being visited. In the first graph there is no loop. After sending the claim back from Surveyor to Assessment, DFS finds a loop.

Task 4: Longest run of consecutive days with more than 100 claims using sliding window

Code:

```
def longest_run(claims):
    left = 0
    best_len = 0
    best_start = 0
    for right in range(len(claims)):
        if claims[right] <= 100:
            left = right + 1          # run is broken, start again
        elif right - left + 1 > best_len:
            best_len = right - left + 1
            best_start = left
    return best_len, best_start

claims = [90, 120, 130, 110, 80, 150, 160, 170, 140, 95, 105]
length, start = longest_run(claims)
print("Longest run:", length, "days, from day", start + 1,
      "to day", start + length)

print("Empty input:", longest_run([])[0])
print("No day above 100:", longest_run([50, 100, 90])[0])
print("All days above 100:", longest_run([101, 150, 200])[0])
```

Output:

```
Longest run: 4 days, from day 6 to day 9
Empty input: 0
No day above 100: 0
All days above 100: 3
```

Explanation:

The window is from left to right. When a day has more than 100 claims, the window grows. When a day has 100 or less, the run is broken, so left moves to the next day. I keep the longest window seen. The answer is 4 days (day 6 to day 9). Empty input gives 0.

Task 5: Merge two sorted claim lists using two pointers

Code:

```
def merge(a, b):
    i = 0
    j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    result.extend(a[i:])      # left-over items of a
    result.extend(b[j:])      # left-over items of b
    return result

office1 = [101, 105, 110, 120]
office2 = [102, 105, 115, 130, 140]
print("Merged:", merge(office1, office2))
print("One list empty:", merge([], [7, 8, 9]))
print("Both empty:", merge([], []))
print("Duplicates:", merge([5, 5, 6], [5, 7]))
```

Output:

```
Merged: [101, 102, 105, 105, 110, 115, 120, 130, 140]
One list empty: [7, 8, 9]
Both empty: []
Duplicates: [5, 5, 5, 6, 7]
```

Explanation:

One pointer is kept on each list. I compare the two current items, put the smaller one in the result and move that pointer. When one list is finished, the left-over items of the other list are added at the end. No extra sorting is needed because both lists are already sorted. Duplicates are kept and empty lists work correctly.

Complexity of each solution

Task	Solution	Time	Space
1	BST insert / search	O(log n) average, O(n) worst	O(n) for the tree
1	Traversals	O(n)	O(h) for recursion (h = height)
2	Hashing for duplicates	O(n) on average	O(n)
3	BFS shortest path and DFS loop check	O(V + E)	O(V)
4	Sliding window	O(n)	O(1)
5	Two-pointer merge	O(n + m)	O(n + m) for the new list

Set 3 – Logistics Network and Delivery Windows

Task 1: BST of hub codes, height of the tree and effect of sorted input

Code:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def insert(root, value):
    if root is None:
        return Node(value)
    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)
    return root

def search(root, value):
    if root is None:
        return False
    if value == root.value:
        return True
    if value < root.value:
        return search(root.left, value)
    return search(root.right, value)

def height(root):
    if root is None:
        return 0
    return 1 + max(height(root.left), height(root.right))

def build(codes):
    root = None
    for code in codes:
        root = insert(root, code)
    return root
```

```
mixed = [50, 30, 70, 20, 40, 60, 80]
sorted_codes = [10, 20, 30, 40, 50, 60, 70]

print("Insert order", mixed, "-> height", height(build(mixed)))
print("Insert order", sorted_codes, "-> height",
      height(build(sorted_codes)))

tree = build(mixed)
print("Search 60:", search(tree, 60))
print("Search 65:", search(tree, 65))
```

Output:

```
Insert order [50, 30, 70, 20, 40, 60, 80] -> height 3
Insert order [10, 20, 30, 40, 50, 60, 70] -> height 7
Search 60: True
Search 65: False
```

Explanation:

Height here means the number of levels in the tree. The same 7 hub codes give height 3 when inserted in mixed order, but height 7 when inserted in sorted order. With sorted input every new code goes to the right side, so the tree becomes like a linked list. Then search takes O(n) time instead of O(log n).

Task 2: Hash map of parcel tracking numbers and check for parcels scanned twice at the same hub

Code:

```
# each scan is (tracking number, hub code)
scans = [("TRK1001", "DEL"), ("TRK1002", "MUM"), ("TRK1001", "DEL"),
         ("TRK1003", "PUN"), ("TRK1002", "CHE"), ("TRK1002", "MUM")]

last_hub = {}       # tracking number -> latest hub
scan_count = {}     # (tracking number, hub) -> how many times scanned

for tracking, hub in scans:
    last_hub[tracking] = hub
    key = (tracking, hub)
    scan_count[key] = scan_count.get(key, 0) + 1

print("Look-up TRK1003:", last_hub.get("TRK1003"))
print("Look-up TRK9999:", last_hub.get("TRK9999", "not found"))

for (tracking, hub), count in scan_count.items():
    if count > 1:
        print(tracking, "scanned", count, "times at", hub)
```

Output:

```
Look-up TRK1003: PUN
Look-up TRK9999: not found
TRK1001 scanned 2 times at DEL
TRK1002 scanned 2 times at MUM
```

Explanation:

The first dictionary stores the latest hub of each tracking number, so look-up is fast. The second dictionary counts scans using (tracking number, hub) as the key. If the count is more than 1, the parcel was scanned twice at the same hub. TRK1002 was scanned at MUM twice, but its scan at CHE is only once, so it is reported only for MUM.

Task 3: 10 hubs as an adjacency list: fewest hops (BFS) and reachable hubs (DFS)

Code:

```
from collections import deque

edges = [("DEL", "JAI"), ("DEL", "LKO"), ("DEL", "MUM"), ("JAI", "AHM"),
         ("AHM", "MUM"), ("MUM", "PUN"), ("PUN", "HYD"), ("PUN", "BLR"),
         ("HYD", "CHE"), ("BLR", "CHE"), ("LKO", "KOL"), ("KOL", "HYD")]

graph = {}
for a, b in edges:
    graph.setdefault(a, []).append(b)
    graph.setdefault(b, []).append(a)

print("Number of hubs:", len(graph))
for hub in graph:
    print(hub, ":", graph[hub])

def fewest_hops(graph, start, end):
    parent = {start: None}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node == end:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for nxt in graph[node]:
            if nxt not in parent:
                parent[nxt] = node
                queue.append(nxt)
    return None

def reachable(graph, node, visited):
    visited.append(node)
    for nxt in graph[node]:
        if nxt not in visited:
            reachable(graph, nxt, visited)

path = fewest_hops(graph, "DEL", "CHE")
print("Fewest hops DEL to CHE:", " -> ".join(path),
      "(" + str(len(path) - 1) + " hops)")

found = []
reachable(graph, "DEL", found)
print("Reachable from DEL:", ", ".join(found))
```

Output:

```
Number of hubs: 10
DEL : ['JAI', 'LKO', 'MUM']
JAI : ['DEL', 'AHM']
LKO : ['DEL', 'KOL']
MUM : ['DEL', 'AHM', 'PUN']
AHM : ['JAI', 'MUM']
PUN : ['MUM', 'HYD', 'BLR']
HYD : ['PUN', 'CHE', 'KOL']
BLR : ['PUN', 'CHE']
CHE : ['HYD', 'BLR']
KOL : ['LKO', 'HYD']
Fewest hops DEL to CHE: DEL -> LKO -> KOL -> HYD -> CHE (4 hops)
Reachable from DEL: DEL, JAI, AHM, MUM, PUN, HYD, CHE, BLR, KOL, LKO
```

Explanation:

The adjacency list stores the direct routes of each hub, and I made every route two-way. BFS is used for the fewest hops because it checks the hubs level by level, so the first path that reaches the destination is the shortest. DEL to CHE needs 4 hops (another route with 4 hops also

exists). DFS from the central depot (DEL) goes as deep as possible and visits every hub that can be reached. Here all 10 hubs are reachable.

Task 4: 4-hour window with the highest average using sliding window

Code:

```
# parcels per hour for 12 hours
parcels = [40, 55, 60, 80, 120, 150, 140, 90, 70, 65, 50, 45]
k = 4

window_sum = sum(parcels[:k])
best_sum = window_sum
best_start = 0

for i in range(k, len(parcels)):
    window_sum = window_sum + parcels[i] - parcels[i - k]
    if window_sum > best_sum:
        best_sum = window_sum
        best_start = i - k + 1

print("Best 4-hour window: hour", best_start, "to hour",
      best_start + k - 1)
print("Average parcels per hour:", best_sum / k)
```

Output:

```
Best 4-hour window: hour 4 to hour 7
Average parcels per hour: 125.0
```

Explanation:

I find the sum of the first 4 hours, and then slide the window by adding the new hour and removing the old hour. The average is the best sum divided by 4. The best window is hours 4 to 7 with an average of 125.0 parcels per hour.

Task 5: Check whether a route description is a palindrome of hub codes using two pointers

Code:

```
import re

def is_route_palindrome(route):
    # split the text at separators and ignore upper/lower case
    codes = []
    for part in re.split(r"[^A-Za-z0-9]+", route):
        if part != "":
            codes.append(part.lower())

    left = 0
    right = len(codes) - 1
    while left < right:
        if codes[left] != codes[right]:
            return False
        left += 1
        right -= 1
    return True

routes = ["DEL-MUM-PUN-MUM-DEL", "del, mum, PUN, Mum, DEL",
          "DEL > MUM > PUN > BLR", "DEL"]
for r in routes:
    print(r, "->", is_route_palindrome(r))
```

Output:

```
DEL-MUM-PUN-MUM-DEL -> True
del, mum, PUN, Mum, DEL -> True
```

```
DEL > MUM > PUN > BLR -> False
DEL -> True
```

Explanation:

First the text is split at the separators (like -, comma, space, >) into hub codes, and the codes are changed to lower case. Then one pointer starts at the first code and one at the last code. If the codes are different it is not a palindrome. If they match, both pointers move towards the middle. If the pointers meet without any mismatch, the route is a palindrome. Here each hub code is treated as one item.